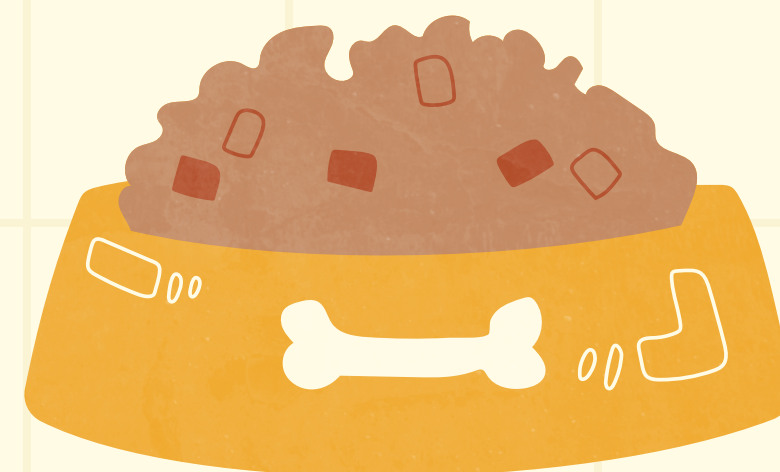
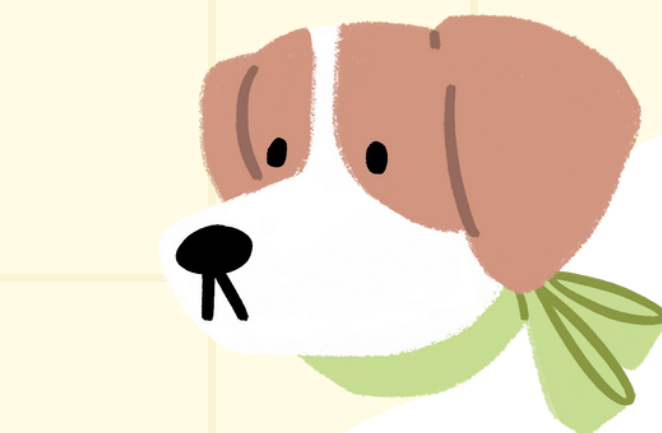
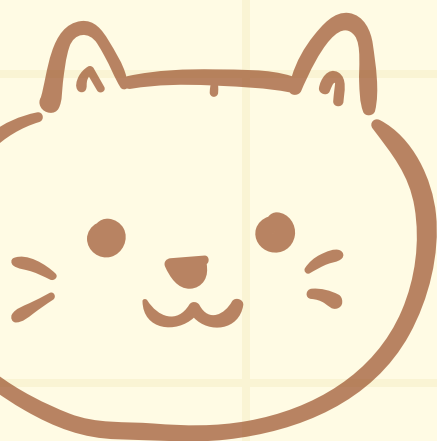


PET SHOP

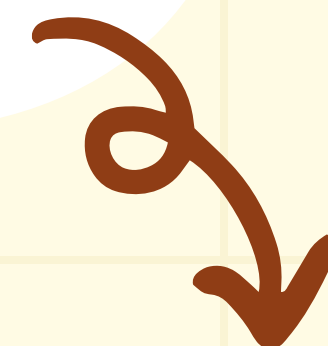
Backend



1. Criar as pastas;
2. Supabase;
3. VSCode;
4. Arquivo .env;
5. Inicie o backend;
6. Server.js;
7. Rotas CRUD.



SUMÁRIO



PASSO A PASSO
DE COMO CRIAR
UM BACKEND



CRIAR AS PASTAS

Para iniciar um projeto Backend é necessário criar um repositório no Github, um arquivo no Supabase (banco de dados) e uma pasta no VSCode.



SUPABASE

TABELA PRODUTO

Antes de começar o código, vamos criar a tabela produtos para armazenar os dados do nosso petshop.

O seguinte código será suficiente para criar a tabela:

```
CREATE TABLE produtos (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  categoria VARCHAR(50),  
  preco DECIMAL(10,2)  
);
```


VSCODE

Coloque a pasta "petshop" no vscode e crie as seguintes

pastas/arquivos:

backend(pasta)

.env (arquivo)

server.js (arquivo)

(Deixe todos os arquivos dentro da pasta "backend")





**ARQUIVO
.ENV**

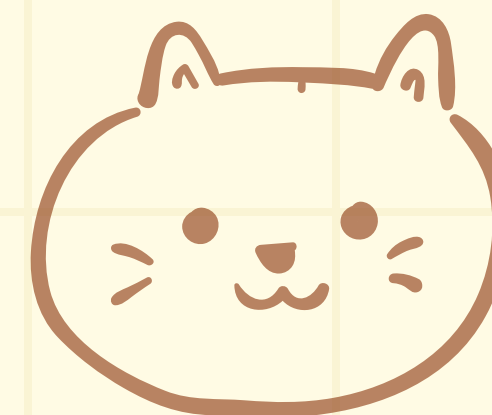


VSCODE



Na pasta .env você irá adicionar os links para conectar o código com sua tabela supabase. Para isso use:

```
SUPABASE_URL=  
SUPABASE_KEY=  
PORT=3000
```





INICIE O BACKEND

Para inicializar o backend você deve abrir um novo terminal, entrar na sua pasta backend (cd backend) e escrever os seguintes comandos:

```
npm init -y  
npm install express
```

(Vai criar arquivos como: package-lock.json e package.json)

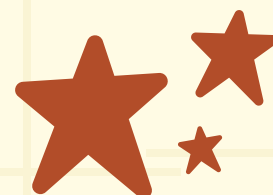


ARQUIVO SERVER.JS

Para criar um backend é preciso criar rotas CRUD como GET, POST, PUT e DELETE. Siga o modelo de código a seguir:

```
const express = require('express');  
const cors = require('cors');  
require('dotenv').config();  
const { createClient } =  
require('@supabase/supabase-js');
```

```
const app = express();  
const PORT = process.env.PORT ||  
3000;
```





MIDDLEWARE

O middleware é um software que diferentes aplicações usam para se comunicar umas com as outras.

CÓDIGO:

```
app.use(cors());  
app.use(express.json());
```



Antes de começar as rotas, vamos inicializar o supabase e criar uma rota teste:

CÓDIGO:

```
// Inicializar Supabase
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);

// Rota teste
app.get('/', (req, res) => {
  res.json({ mensagem: '● API rodando!' });
});
```



**BUILDING BONDS
BEYOND THE LEASH!**



**INICIALIZAR
E ROTA
TESTE**



ROTA GET

Buscar todos os produtos

CÓDIGO:

```
app.get('/produtos', async (req, res) => {  
  try {  
    const { data, error } = await supabase  
      .from('produtos')  
      .select('*');  
    if (error) throw error;  
    res.json(data);  
  } catch (error) {  
    res.status(400).json({ erro: error.message });  
  }  
});
```



ROTA GET

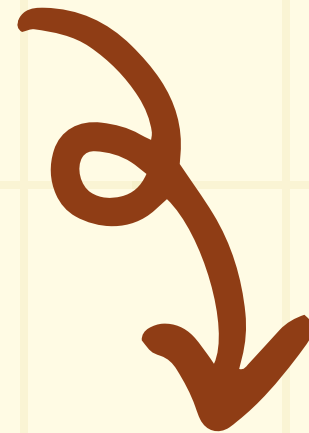
Buscar produto por ID

```
app.get('/produtos/:id', async (req, res) => {  
  try {  
    const { data, error } = await supabase  
      .from('produtos')  
      .select('*')  
      .eq('id', req.params.id)  
      .single();  
    if (error) throw error;  
    res.json(data);  
  } catch (error) {  
    res.status(400).json({ erro: error.message });  
  }  
});
```





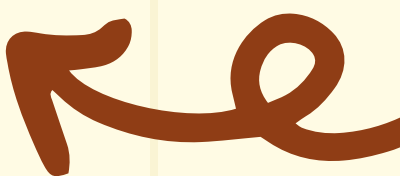
CRIAR NOVO
PRODUTO



ROTA
POST



```
app.post('/produtos', async (req, res) => {  
  try {  
    const { nome, especie, categoria, descricao, preco } = req.body;  
    const { data, error } = await supabase  
      .from('produtos')  
      .insert([ { nome, especie, categoria, descricao, preco } ])  
      .select();  
    if (error) throw error;  
    res.json(data);  
  } catch (error) {  
    res.status(400).json({ erro: error.message });  
  }  
});
```



ROTA PUT

```
app.put('/produtos/:id', async (req, res) => {  
  try {  
    const { nome, especie, categoria, descricao, preco } = req.body;  
    const { data, error } = await supabase  
      .from('produtos')  
      .update({ nome, especie, categoria, descricao, preco })  
      .eq('id', req.params.id)  
      .select();  
    if (error) throw error;  
    res.json(data);  
  } catch (error) {  
    res.status(400).json({ erro: error.message });  
  }  
});
```





ROTA DELETE

```
app.delete('/produtos/:id', async (req, res) => {  
  try {  
    const { error } = await supabase  
      .from('produtos')  
      .delete()  
      .eq('id', req.params.id);  
    if (error) throw error;  
    res.json({ mensagem: '✗ Produto deletado!' });  
  } catch (error) {  
    res.status(400).json({ erro: error.message });  
  }  
});
```

```
app.listen(PORT, () => {  
  console.log(`Servidor rodando em http://localhost:${PORT}`);  
});
```

The background of the entire image is a light yellow color with faint, repeating outlines of cat faces. A thick, brown, wavy line runs horizontally across the upper third of the image. On the left side, there is a vertical stack of two cartoon dogs: a brown one on top and a tan one on the bottom, both wearing pink scarves. On the right side, there is a vertical stack of two cartoon dogs: a brown one on top and a white one on the bottom, both wearing blue bows. In the center, there is a large white rectangular area containing text and a code editor interface.

Instruções

THUNDER CLIENT

Agora vamos instalar a extensão “Thunder Client” e criar um “New Request”. Para testar as rotas, antes é preciso iniciar o servidor, então abra um novo terminal e digite: `node server.js` (Verifique que você está na pasta correta).
Copie o link que apareceu e cole aqui:

GET

<https://www.thunderclient.com/welcome>

Send

TESTE ROTA POST

Vamos adicionar novos produtos que apareceram na tabela Supabase automaticamente:

```
{  
  "nome": " ",  
  "especie": " ",  
  "categoria": " ",  
  "descricao": " ",  
  "preco":  
}
```

**ADICIONEI
COLUNA
(DESCRICAO)
NA TABELA
SUPABASE**

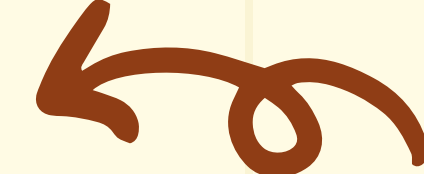
Clique no GET e mude a rota para POST, verifique que o link está assim:
<http://localhost:3000/produtos>

Adicione o código no BODY.





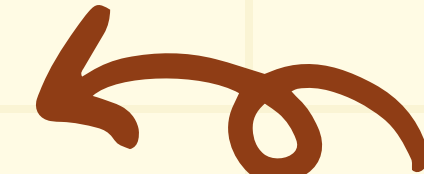
Mude o POST para GET e clique em SEND, todos os produtos adicionados vão aparecer para você.



**TESTAR
ROTA GET**



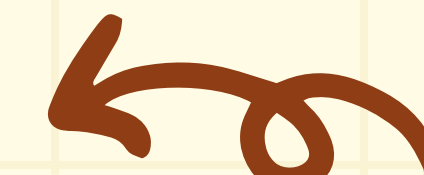
Mude o GET para PUT e clique em SEND, adicione um /id (no lugar do id coloque o número que você deseja alterar). No body escreva o mesmo código do slide anterior e faça sua alteração, depois clique no SEND e veja a mudança.



**TESTAR
ROTA PUT**



Mude o PUT para DELETE, no link acima adicione o id do produto e clique em SEND. O produto irá ser deletado do VSCode e da Tabela.



**TESTAR
ROTA DELETE**



The image features a central white cloud-like shape with a brown outline, containing the word "OBRIGADA" in bold brown capital letters. Four cartoon dogs are positioned around the shape: a brown dog with floppy ears and a pink bow at the top left; a brown and white dog with long ears and a bell collar at the top right; a white dog with brown patches and a green collar holding a blue ball at the bottom left; and a fluffy brown dog with a blue bow at the bottom right. The background is a light yellow color with faint, repeating outlines of cat faces and paw prints. A dashed orange line follows the path of the dogs around the central text.

OBRIGADA